实验报告

一、实验目的

本次实验旨在研究并分析不同网络结构和训练参数对神经网络训练效果的影响，主要包括：

1. **隐含层节点数目对训练精度的影响。**
2. **不同梯度更新步长对训练过程的影响。**
3. **在固定网络结构的情况下，目标函数随着迭代步数变化的曲线。**

二、实验方法与流程

1.构建了一个三层神经网络。

该网络由输入层、一个隐含层和输出层组成。网络的前向传播过程如下：

1. **输入层**接受输入数据，通过与输入层到隐含层的权重矩阵相乘，得到隐含层的输入。
2. **隐含层**的输出通过 `tanh` 激活函数计算，得到隐含层的输出。
3. **输出层**的输入通过隐含层输出与隐含层到输出层的权重矩阵相乘，并通过 `sigmoid` 激活函数计算，得到最终的网络输出。

2.反向传播与权重更新

网络的训练使用反向传播算法。具体步骤如下：

1. 计算**输出误差**，即实际输出与目标输出之间的差异。
2. 计算**隐含层与输出层之间的权重更新**，并根据误差调整权重。
3. 使用梯度下降法更新权重，调整网络参数以最小化误差。

我们采用两种不同的训练方式：

- **单样本更新（Stochastic Gradient Descent, SGD）**：每次迭代仅使用一个样本来更新权重。
- **批量更新（Batch Gradient Descent）**：每次迭代使用整个训练集的平均误差来更新权重。

```
class net:
    """
    三层网络类
    """
    def __init__(self, train_data, train_label, h_num):
        """
        网络初始化
        Parameters:
            train_data: 训练用数据列表
            train_label: 训练用Label列表
            h_num: 隐含层结点数
        """
        # 初始化数据
        self.train_data = train_data
        self.train_label = train_label
        self.h_num = h_num
        # 随机初始化权重矩阵
        self.w_ih = np.random.rand(train_data[0].shape[0], h_num)
```

```python
        self.w_hj = np.random.rand(h_num, train_label[0].shape[0])

    def tanh(self, data):
        """
        tanh函数
        """
        return (np.exp(data) - np.exp(-data)) / (np.exp(data) + np.exp(-data))

    def sigmoid(self, data):
        """
        Sigmoid函数
        """
        return 1 / (1 + np.exp(-data))

    def forward(self, data):
        """
        前向传播
        Parameter:
            data: 单个样本输入数据
        Return:
            z_j: 单个输入数据对应的网络输出
            y_h: 对应的隐含层输出，用于后续反向传播时权重更新矩阵的计算
        """
        # 计算隐含层输出
        net_h = np.matmul(data.T, self.w_ih)
        y_h = self. tanh(net_h)
        # 计算输出层输出
        net_j = np.matmul(y_h.T, self.w_hj)
        z_j = self.sigmoid(net_j)

        return z_j, y_h

    def backward(self, z, label, eta, y_h, x_i):
        """
        反向传播
        Parameters:
            z: 前向传播计算的网络输出
            label: 对应的Label
            eta: 学习率
            y_h: 对应的隐含层输出
            x_i: 对应的输入数据
        Return:
            delta_w_hj: 隐含层-输出层权重更新矩阵
            delta_w_ih: 输入层-隐含层权重更新矩阵
            error: 样本输出误差，用于后续可视化
        """
        # 矩阵维度整理
        z = np.reshape(z, (z.shape[0], 1))
        label = np.reshape(label, (label.shape[0], 1))
        y_h = np.reshape(y_h, (y_h.shape[0], 1))
        x_i = np.reshape(x_i, (x_i.shape[0], 1))
        # 计算输出误差
        error = np.matmul((label-z).T, (label-z))[0][0]
        # 计算隐含层-输出层权重更新矩阵
        error_j = (label - z) * z * (1-z)
        delta_w_hj = eta * np.matmul(y_h, error_j.T)
```

```
        # 计算输入层-隐含层权重更新矩阵
        error_h = np.matmul(((label - z) * z * (1-z)).T, self.w_hj.T).T * (1-
y_h**2)
        delta_w_ih = eta * np.matmul(x_i, error_h.T)

        return delta_w_hj, delta_w_ih, error

    def train(self, bk_mode, eta, epoch_num):
        """
        网络训练
        Parameters:
            bk_mode: 反向传播方式('single' or 'batch')
            eta: 学习率
            epoch_num: 全部训练数据迭代次数
        """
        # 单样本更新
        if bk_mode == 'single':
            E = []
            for _ in range(epoch_num):
                e = []
                for idx, x_i in enumerate(self.train_data):
                    # 前向传播
                    z, y_h = self.forward(x_i)
                    # 反向传播
                    delta_w_hj, delta_w_ih, error = self.backward(z,
self.train_label[idx], eta, y_h, x_i)
                    # 权重矩阵更新
                    self.w_hj += delta_w_hj
                    self.w_ih += delta_w_ih

                    e.append(error)
                E.append(np.mean(e))

        # 批次更新
        if bk_mode == 'batch':
            E = []
            for _ in range(epoch_num):
                e = []
                Delta_w_hj = 0
                Delta_w_ih = 0
                for idx, x_i in enumerate(self.train_data):
                    # 前向传播
                    z, y_h = self.forward(x_i)
                    # 反向传播
                    delta_w_hj, delta_w_ih, error = self.backward(z,
self.train_label[idx], eta, y_h, x_i)
                    # 更新权重矩阵累加
                    Delta_w_hj += delta_w_hj
                    Delta_w_ih += delta_w_ih

                    e.append(error)
                # 权重矩阵批次更新
                self.w_hj += Delta_w_hj
                self.w_ih += Delta_w_ih
                E.append(np.mean(e))
```

```
        # 可视化迭代优化过程
        import matplotlib.pyplot as plt
        plt.plot(E)
        plt.show()
```

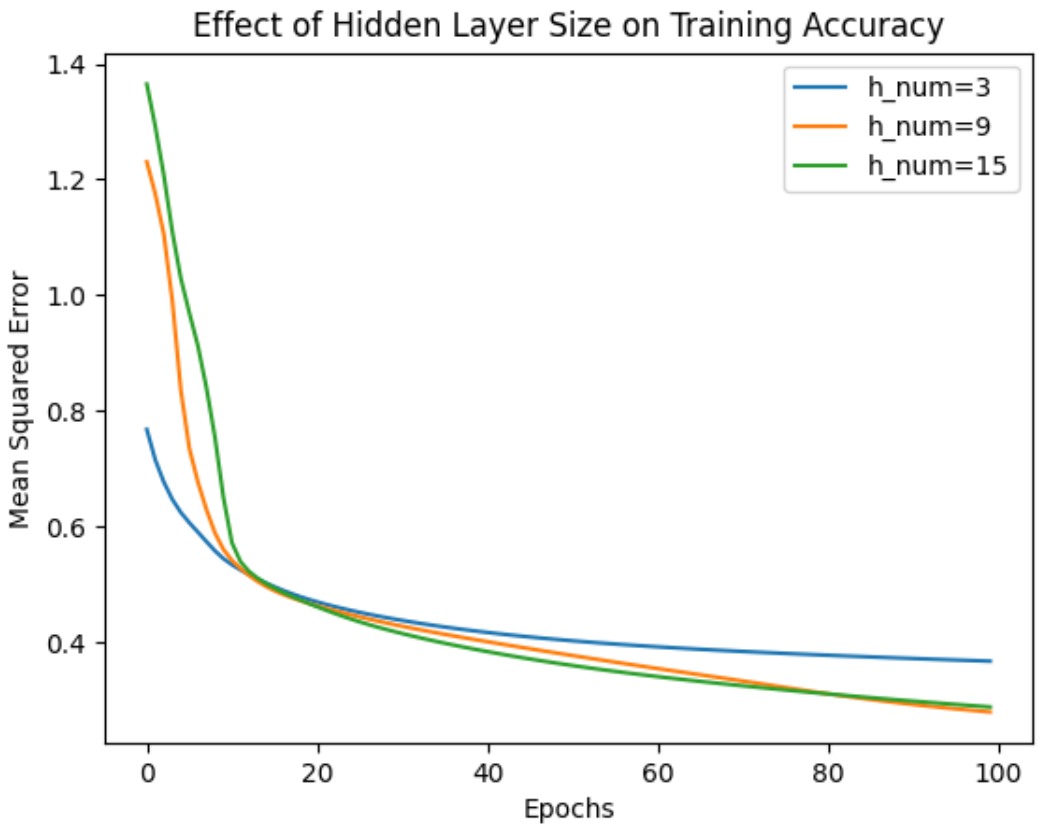
三、实验分析

1. 隐含层节点数对训练精度的影响

我们通过改变隐含层的节点数（3、9、15），观察误差随迭代次数的变化曲线。结果显示：

- **较少的隐含层节点（如3个）**时，网络的拟合能力较差，误差下降较慢，可能无法捕捉到数据的复杂模式。
- **中等数量的隐含层节点（如9个）**时，网络的拟合效果较好，误差随迭代次数减小。
- **较多的隐含层节点（如15个）**时，误差仍然较小，但过多的隐含层节点可能导致计算效率的下降，并且可能出现过拟合的风险。

结果图示：

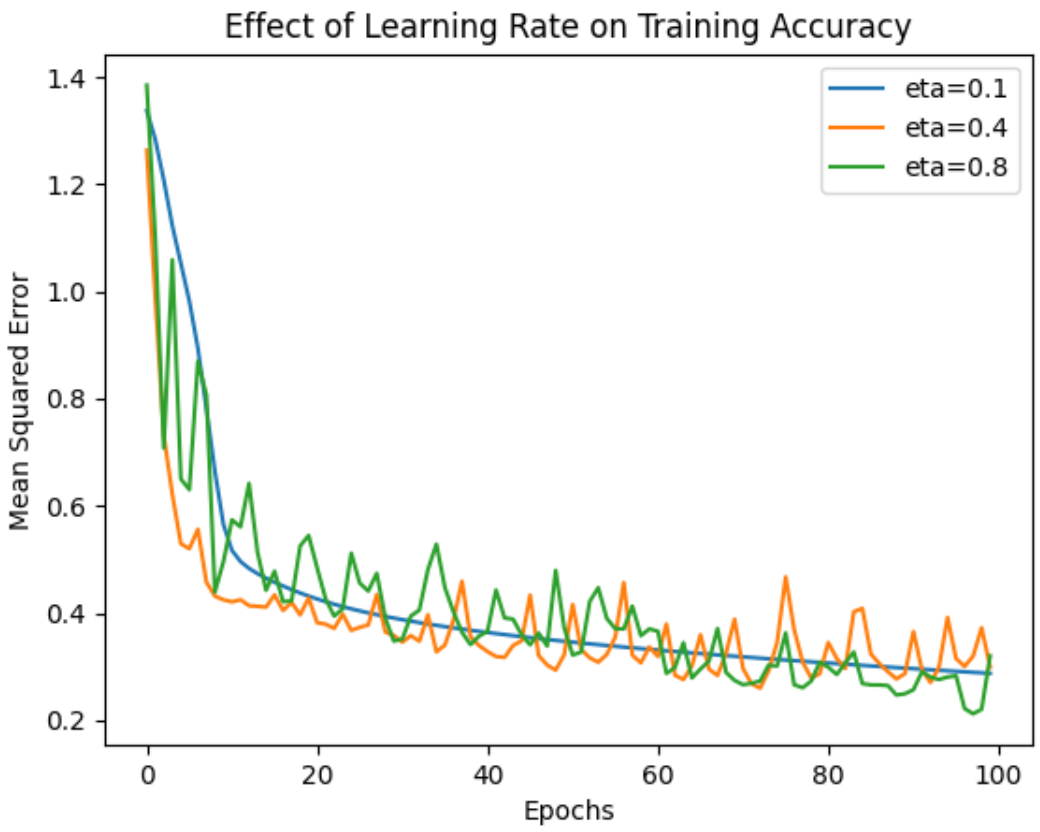


2. 学习率对训练精度的影响

我们分别设定不同的学习率（0.1、0.4、0.8），使用批量更新方式训练网络。结果表明：

- **较小的学习率（如0.1）**时，误差下降较为缓慢，可能需要较多的迭代次数才能达到较低的误差值。
- **中等的学习率（如0.4）**时，网络的收敛速度较快，误差较快下降到较低的值。
- **较大的学习率（如0.8）**时，网络可能会发生震荡，误差在一些迭代中可能增大，表现出不稳定的收敛行为。

结果图示:



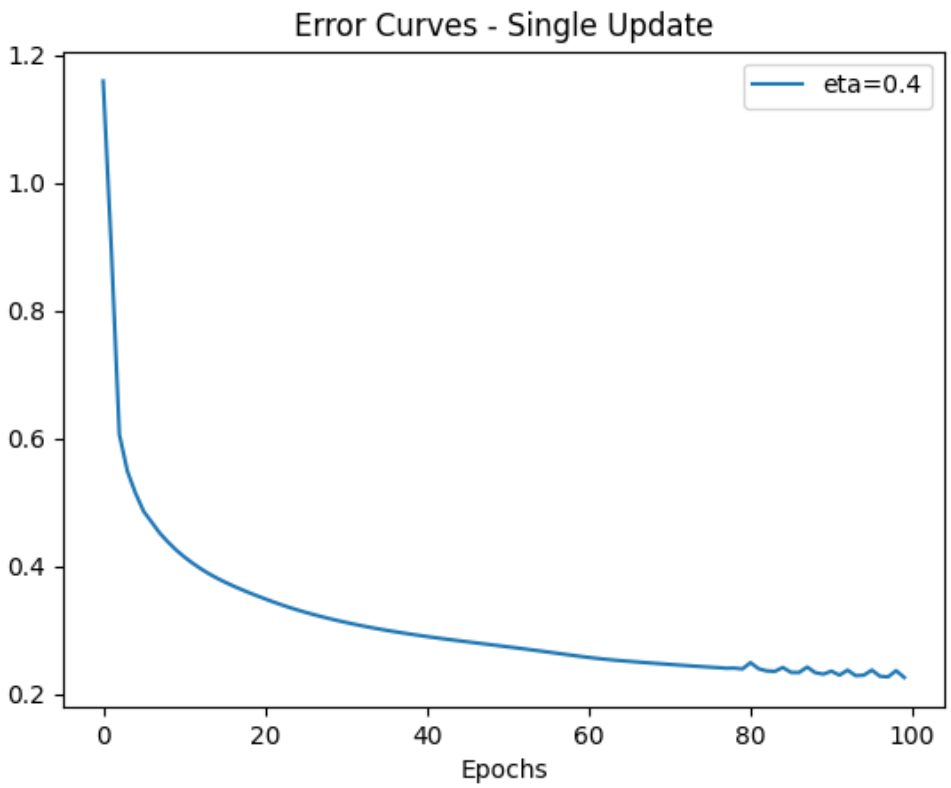
3. 更新策略对训练过程的影响

我们分别使用单样本更新和批量更新训练模型，观察训练误差的变化。结果显示:

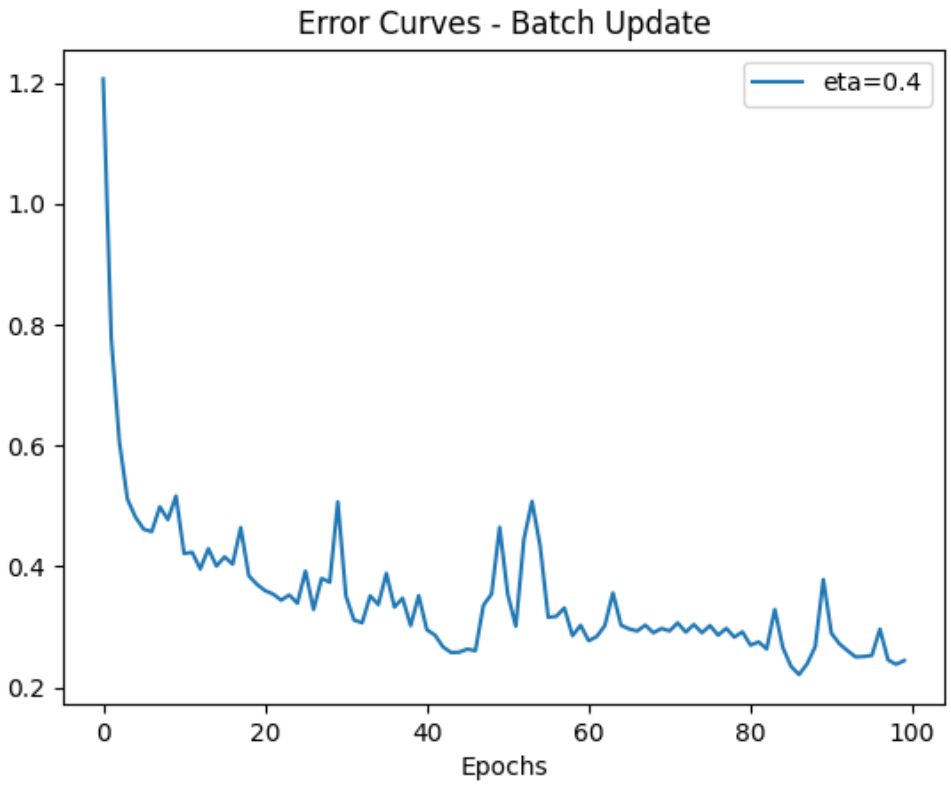
- **单样本更新（SGD）**训练过程较为平稳, 曲线较为光滑, 最终的训练误差在0.05左右。
- **批量更新（Batch Gradient Descent）**相同参数下, 批量更新方式下的曲线是震荡下降的. 若希望得到平稳的训练过程, 则需要适量减小更新步长

结果图示:

- 单样本更新误差曲线:



- 批量更新误差曲线：



四、结论

1. **隐含层节点数**：适当增加隐含层的节点数可以提高网络的拟合能力，但节点数过多可能会导致过拟合和计算效率下降。
2. **学习率**：合适的学习率能加快训练过程，并帮助网络稳定收敛。过小的学习率会导致收敛过慢，而过大的学习率会导致震荡。

3. **更新策略**：单样本更新（SGD）训练过程较为平稳, 曲线较为光滑,批量更新（Batch Gradient Descent）相同参数下, 曲线是震荡下降的.，若希望得到平稳的训练过程, 则需要适量减小更新步长

本实验提供了神经网络训练过程中的超参数调优的一些见解，未来可以进一步探索其他优化算法（如Adam、RMSprop）对训练过程的影响。